Biblioteca Tecnológica — MOBILE (Kotlin + Jetpack Compose)

Documento de estudo para a apresentação de Mobile. Explica o que o aplicativo faz, como ele conversa com o Backend e cada parte do código.

---

1. O que o aplicativo faz (em uma frase)

É um app Android que **mostra os livros da biblioteca**, **busca um livro pelo ID** e **cadastra um livro novo**. Todos os dados vêm do Backend (Spring Boot) pela internet. O app não guarda livros por conta própria.

O que aparece na tela (de cima para baixo)

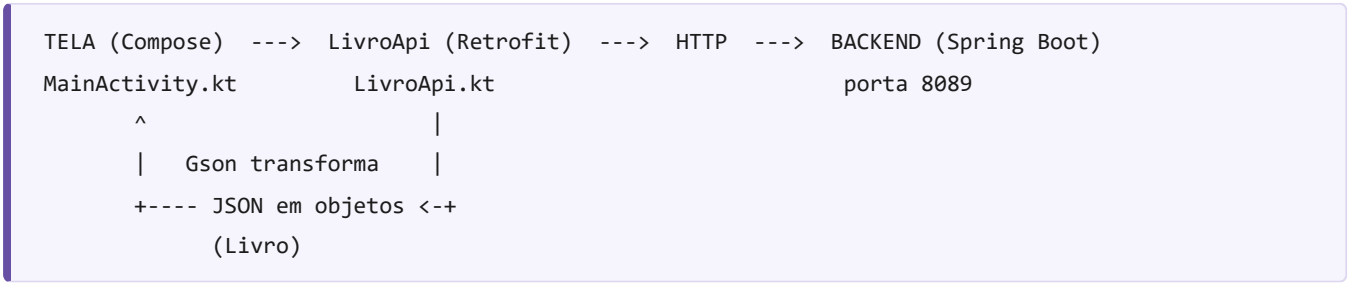
1. **Título:** "Biblioteca Tecnológica".
2. **Busca:** campo "ID do livro" + botão **Buscar**. Mostra o livro ou "Livro não encontrado.".
3. **"Todos os livros":** carrossel (uma fileira que rola para o lado) com todos os livros.
4. **Cadastrar livro:** campo Título, caixa **Disponível**, escolha da capa e botão **Cadastrar**.

Ligação entre a tela e o Backend

Ação do usuário	Chamada HTTP	Resultado na tela
Abrir o app	`GET /livros`	O carrossel é preenchido
Digitar um ID e tocar em Buscar	`GET /livros/{id}`	Aparece o card do livro, ou "Livro não encontrado." (404)
Preencher e tocar em Cadastrar	`POST /livros`, depois `GET /livros`	Mensagem de sucesso e carrossel atualizado

---

2. Como o app e o Backend se comunicam



Os papéis:

Peça	Arquivo	Papel
Modelos	`LivroModels.kt`	Classes que representam os dados (`Livro` , `LivroRequest`)
API	`LivroApi.kt`	Diz **quais chamadas** o app faz e **para onde** (Retrofit)

Peça	Arquivo	Papel
Telas	`MainActivity.kt`	Desenha a tela e reage aos toques
Permissões	`AndroidManifest.xml`	Autoriza o app a usar a internet

Endereço do Backend: `http://127.0.0.1:8089/`

- O emulador é um "celular de mentira" dentro do computador. Para o emulador, `127.0.0.1` é **ele mesmo**, e não o computador.
- Por isso usamos o **ADB Reverse**, que **encaminha** a porta 8089 do emulador para a porta 8089 do computador: `adb reverse tcp:8089 tcp:8089`
- O app chama `127.0.0.1:8089` e o ADB leva o pedido até o Backend.
- Se o emulador for reiniciado, o comando precisa ser executado de novo.
- A alternativa era `10.0.2.2`, o endereço que o emulador usa para falar com o computador. Neste projeto, o app usa `127.0.0.1` com o ADB Reverse.

---

3. Estrutura dos arquivos

```
ANDROID/app/src/main
├── AndroidManifest.xml
├── java/com/example/android
│   ├── MainActivity.kt      (tela e lógica)
│   ├── LivroApi.kt          (Retrofit)
│   └── LivroModels.kt       (classes de dados e capas)
└── res/drawable             (as imagens das capas)
    ├── aneis.jpg  harry.jpg  dom.jpg  orwel.jpg  principe.png
```

---

4. `AndroidManifest.xml` — as autorizações

```
<uses-permission android:name="android.permission.INTERNET" />

<application
    ...
    android:usesCleartextTraffic="true"
    ...>
    <activity
        android:name=".MainActivity"
        android:exported="true"
        ...>
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

Trecho	Explicação
`uses-permission ... INTERNET`	Sem essa linha, o Android **bloqueia** qualquer acesso à rede.
`usesCleartextTraffic="true"`	Permite **HTTP** simples (sem `https`). O Backend de estudo não tem certificado, então precisamos disso no ambiente de desenvolvimento.
`<activity android:name=".MainActivity">`	Declara a tela (Activity) do app.
`exported="true"` + `intent-filter` MAIN/LAUNCHER	Faz a `MainActivity` ser a **tela inicial**, aquela que abre ao tocar no ícone.

5. `LivroModels.kt` — os dados

```
package com.example.android

data class Livro(
    val id: Long,
    val titulo: String,
    val disponivel: Boolean,
    val capa: String,
)

data class LivroRequest(
    val titulo: String,
    val disponivel: Boolean,
    val capa: String,
)

fun capaParaImagem(capa: String): Int = when (capa) {
    "aneis" -> R.drawable.aneis
    "harry" -> R.drawable.harry
    "dom" -> R.drawable.dom
    "orwel" -> R.drawable.orwel
    "principe" -> R.drawable.principe
    else -> R.drawable.dom
}

val codigosDeCapa = listOf("aneis", "harry", "dom", "orwel", "principe")
```

Trecho	Explicação
`data class Livro(...)`	Classe **só para guardar dados**. O Kotlin cria sozinho `toString`, `equals`, etc.
`val id: Long`	`val` = valor que **não muda**. O tipo vem depois dos dois pontos.
`Livro`	É o que **chega** do Backend (tem `id`). Os nomes dos campos são **iguais** aos do JSON, e é assim que o Gson consegue converter.
`LivroRequest`	É o que o app **envia** no POST. **Não tem** `id`, porque o Backend gera.
`fun capaParaImagem(capa: String): Int`	Recebe o código da capa (texto) e devolve o **número** da imagem (`Int`). No Android, cada imagem de `res/drawable` ganha um número em `R.drawable`.
`when (capa) { ... }`	É o "se/senão" do Kotlin com várias opções. Se for `"aneis"`, usa `R.drawable.aneis`, e assim por diante.
`else -> R.drawable.dom`	Se vier uma capa desconhecida, usa uma imagem padrão.
`val codigosDeCapa = listOf(...)`	A lista das 5 capas, usada para mostrar as opções no formulário.

Para explicar: *"O Backend guarda só o texto da capa. A imagem está dentro do app. A função `capaParaImagem` liga um ao outro."*

---

6. `LivroApi.kt` — a conversa com o Backend (Retrofit)

```
package com.example.android

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.Body
import retrofit2.http.GET
import retrofit2.http.POST
import retrofit2.http.Path

interface LivroApi {

    @GET("livros")
    suspend fun listarLivros(): List<Livro>

    @GET("livros/{id}")
    suspend fun buscarLivro(@Path("id") id: Long): Livro

    @POST("livros")
    suspend fun cadastrarLivro(@Body request: LivroRequest): Livro
}

object ApiClient {
    private const val URL_BASE = "http://127.0.0.1:8089/"

    val livroApi: LivroApi = Retrofit.Builder()
        .baseUrl(URL_BASE)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
        .create(LivroApi::class.java)
}
```

A interface `LivroApi`

Trecho	Explicação
Retrofit	Biblioteca que transforma uma **interface Kotlin** em chamadas HTTP. Só descrevemos; o Retrofit escreve o código.
`@GET("livros")`	Faz um **GET** em `http://127.0.0.1:8089/livros`.
`suspend fun`	Função que **pode demorar** (rede). Roda dentro de uma **coroutine** e **não trava a tela** enquanto espera.
`listarLivros(): List<Livro>`	Devolve uma **lista de** `Livro`. O Gson converte o JSON em objetos.
`@GET("livros/{id}")`	GET com uma parte variável no endereço.
`@Path("id") id: Long`	Coloca o valor de `id` no lugar de `{id}`. Com `3` o endereço fica `/livros/3`.

Trecho	Explicação
`@POST("livros")`	Faz um **POST** em `/livros`.
`@Body request: LivroRequest`	O objeto vira o **corpo JSON** do pedido.
`cadastrarLivro(...): Livro`	O Backend responde com o livro criado (já com `id`).

O objeto `ApiClient`

Trecho	Explicação
`object ApiClient`	`object` cria **uma única instância** no app todo (singleton). O Retrofit é criado uma vez só.
`private const val URL_BASE`	O endereço base. **Precisa terminar com `/`**, ou o Retrofit dá erro.
`Retrofit.Builder()`	Começa a montar o Retrofit.
`.baseUrl(URL_BASE)`	Define o endereço do Backend.
`.addConverterFactory(GsonConverterFactory.create())`	Liga o **Gson**: converte JSON em objeto Kotlin (e o contrário).
`.build()`	Termina de montar.
`.create(LivroApi::class.java)`	Cria a **implementação** da interface `LivroApi`.
`val livroApi`	É o que as telas usam: `ApiClient.livroApi.listarLivros()`.

---

7. `MainActivity.kt` — a tela

7.1 Conceitos do Jetpack Compose (leia antes)

Conceito	Explicação simples
Compose	Em vez de desenhar a tela em XML, **escrevemos funções** que descrevem a tela.
`@Composable`	Marca uma função que **desenha uma parte da tela**.
Estado (`mutableStateOf`)	Um valor que, **quando muda, faz a tela se redesenhar sozinha**.
`remember`	Faz o Compose **lembrar** do valor entre os redesenhos (se não, ele voltava ao início).
`by`	Permite usar `titulo` direto, em vez de `titulo.value`.
`Column` / `Row`	Colocam os itens **em coluna** (um embaixo do outro) ou **em linha**.
`LazyRow`	Uma linha que **rola para o lado** (o carrossel).
`Modifier`	Ajustes de aparência: espaçamento (`padding`), largura, altura...

Conceito	Explicação simples
Callback (`onBuscar` , `onCadastrar`)	Uma função que **a tela pai entrega** para o componente chamar quando o botão for tocado.

7.2 A classe principal

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            TelaBiblioteca()
        }
    }
}
```

Trecho	Explicação
`class MainActivity : ComponentActivity()`	É a **tela do app** (uma Activity).
`override fun onCreate(...)`	O Android chama esta função quando a tela é criada.
`super.onCreate(...)`	Executa a preparação padrão do Android.
`setContent { TelaBiblioteca() }`	Define **o que aparece**: a função `TelaBiblioteca` .

7.3 `CardLivro` — o cartão de um livro

```
@Composable
fun CardLivro(livro: Livro) {
    Card(
        modifier = Modifier
            .padding(8.dp)
            .width(150.dp)
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Image(
                painter = painterResource(id = capaParaImagem(livro.capa)),
                contentDescription = livro.titulo,
                modifier = Modifier
                    .fillMaxWidth()
                    .height(180.dp),
                contentScale = ContentScale.Crop,
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = livro.titulo,
                style = MaterialTheme.typography.titleSmall,
            )
            Spacer(modifier = Modifier.height(4.dp))
            Text(
                text = if (livro.disponivel) "Disponível" else "Indisponível",
                style = MaterialTheme.typography.bodySmall,
                color = if (livro.disponivel) Color(0xFF2E7D32) else Color(0xFFC62828),
            )
        }
    }
}
```

Trecho	Explicação
`fun CardLivro(livro: Livro)`	Recebe **um livro** e desenha o cartão dele.
`Card(...)`	O cartão (fundo arredondado).
`.padding(8.dp)` / `.width(150.dp)`	Espaço externo de 8 e largura fixa de 150. `dp` é a unidade de medida do Android.
`Column`	Os itens do cartão ficam um embaixo do outro.
`Image(painter = painterResource(...))`	Mostra uma imagem. `capaParaImagem(livro.capa)` descobre **qual** imagem a partir do código.
`contentDescription`	Texto de acessibilidade (leitores de tela).
`ContentScale.Crop`	A imagem **preenche** o espaço, cortando as bordas se precisar.
`Spacer`	Espaço em branco entre os itens.

Trecho	Explicação
`Text(text = livro.titulo)`	O título do livro.
`if (livro.disponivel) "Disponível" else "Indisponível"`	O texto muda conforme o valor. Em Kotlin o `if` **devolve um valor**.
`color = ... Color(0xFF2E7D32) ... Color(0xFFC62828)`	**Verde** quando disponível, **vermelho** quando indisponível.

7.4 `CarrosselDeLivros` — a fileira que rola

```
@Composable
fun CarrosselDeLivros(listaDeLivros: List<Livro>) {
    LazyRow(
        modifier = Modifier.padding(16.dp),
    ) {
        items(listaDeLivros) { livroAtual ->
            CardLivro(livro = livroAtual)
        }
    }
}
```

Trecho	Explicação
`listaDeLivros: List<Livro>`	Recebe a lista pronta. O carrossel **não** busca nada: só desenha.
`LazyRow`	Linha que rola na horizontal e só desenha o que está visível.
`items(listaDeLivros) { livroAtual -> ... }`	**Para cada livro** da lista, desenha um `CardLivro`. É como um `for`.

7.5 `BuscaLivro` — o campo de busca

```
@Composable
fun BuscaLivro(onBuscar: (String) -> Unit) {
    var id by remember { mutableStateOf("") }

    Row(
        modifier = Modifier.padding(horizontal = 16.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        OutlinedTextField(
            value = id,
            onValueChange = { id = it },
            label = { Text("ID do livro") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            modifier = Modifier.weight(1f),
        )
        Spacer(modifier = Modifier.width(8.dp))
        Button(onClick = { onBuscar(id) }) {
            Text("Buscar")
        }
    }
}
```

Trecho	Explicação
`onBuscar: (String) -> Unit`	Um **parâmetro que é uma função**: recebe um texto e não devolve nada. A tela principal entrega o que fazer na busca.
`var id by remember { mutableStateOf("") }`	O texto digitado, guardado como **estado**, começando vazio.
`Row`	Campo e botão **lado a lado**.
`verticalAlignment = CenterVertically`	Alinha os dois no meio, na vertical.
`OutlinedTextField`	Campo de texto com borda.
`value = id`	O que o campo mostra.
`onValueChange = { id = it }`	A cada tecla digitada, atualiza o estado (`it` é o novo texto).
`label = { Text("ID do livro") }`	Rótulo do campo.
`singleLine = true`	Uma linha só.
`keyboardType = KeyboardType.Number`	Abre o **teclado numérico**.
`Modifier.weight(1f)`	O campo ocupa **todo o espaço que sobra** na linha.
`Button(onClick = { onBuscar(id) })`	Ao tocar, chama a função recebida **passando o texto digitado**.

7.6 `FormularioCadastro` — o cadastro

```
@Composable
fun FormularioCadastro(onCadastrar: (LivroRequest) -> Unit) {
    var titulo by remember { mutableStateOf("") }
    var disponivel by remember { mutableStateOf(true) }
    var capa by remember { mutableStateOf(codigosDeCapa.first()) }

    Column(modifier = Modifier.padding(horizontal = 16.dp)) {
        Text("Cadastrar livro", style = MaterialTheme.typography.titleMedium)
        Spacer(modifier = Modifier.height(8.dp))

        OutlinedTextField(
            value = titulo,
            onValueChange = { titulo = it },
            label = { Text("Título") },
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
        )

        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(
                checked = disponivel,
                onCheckedChange = { disponivel = it },
            )
            Text("Disponível")
        }

        Text("Escolha a capa", style = MaterialTheme.typography.labelLarge)
        LazyRow {
            items(codigosDeCapa) { codigo ->
                val selecionada = codigo == capa
                Card(
                    modifier = Modifier
                        .padding(4.dp)
                        .clickable { capa = codigo },
                    border = if (selecionada) {
                        BorderStroke(3.dp, MaterialTheme.colorScheme.primary)
                    } else {
                        null
                    },
                ) {
                    Image(
                        painter = painterResource(id = capaParaImagem(codigo)),
                        contentDescription = codigo,
                        modifier = Modifier.size(width = 60.dp, height = 80.dp),
                        contentScale = ContentScale.Crop,
                    )
```

```
                }
            }
        }
        Spacer(modifier = Modifier.height(8.dp))

        Button(
            onClick = { onCadastrar(LivroRequest(titulo.trim(), disponivel, capa)) },
            modifier = Modifier.fillMaxWidth(),
        ) {
            Text("Cadastrar")
        }
    }
}
```

Trecho	Explicação
`onCadastrar: (LivroRequest) -> Unit`	Função recebida: quando o botão for tocado, ela recebe o livro montado.
`var titulo ... mutableStateOf("")`	Estado do título (começa vazio).
`var disponivel ... mutableStateOf(true)`	Estado do Checkbox (começa **marcado**).
`var capa ... mutableStateOf(codigosDeCapa.first())`	Capa escolhida. Começa na primeira (`aneis`).
`Checkbox(checked, onCheckedChange)`	Caixinha marcada/desmarcada. Ao tocar, atualiza `disponivel` (`it` = o novo valor).
`LazyRow { items(codigosDeCapa) { ... } }`	Mostra **as 5 capas** em uma fileira.
`val selecionada = codigo == capa`	`true` se esta capa for a escolhida agora.
`.clickable { capa = codigo }`	Ao tocar na capa, ela vira a escolhida.
`border = if (selecionada) BorderStroke(...) else null`	**Borda roxa** só na capa escolhida; `null` = sem borda.
`Modifier.size(width = 60.dp, height = 80.dp)`	Tamanho das miniaturas.
`LivroRequest(titulo.trim(), disponivel, capa)`	Monta o objeto a enviar. `trim()` tira espaços do começo e do fim do título.
`onCadastrar(...)`	Entrega o livro montado para a tela principal.

7.7 `TelaBiblioteca` — o cérebro da tela

```
@Composable
fun TelaBiblioteca() {
    var livros by remember { mutableStateOf<List<Livro>>(emptyList()) }
    var livroEncontrado by remember { mutableStateOf<Livro?>(null) }
    var mensagemBusca by remember { mutableStateOf("") }
    var mensagemCadastro by remember { mutableStateOf("") }
    val escopo = rememberCoroutineScope()

    suspend fun carregarLivros() {
        try {
            livros = ApiClient.livroApi.listarLivros()
        } catch (e: Exception) {
            mensagemBusca = "Erro ao carregar livros: ${e.message}"
        }
    }

    LaunchedEffect(Unit) { carregarLivros() }

    Column(modifier = Modifier.verticalScroll(rememberScrollState())) {
        Text(
            text = "Biblioteca Tecnológica",
            style = MaterialTheme.typography.headlineSmall,
            modifier = Modifier.padding(16.dp),
        )

        BuscaLivro(onBuscar = { texto ->
            val id = texto.toLongOrNull()
            if (id == null) {
                livroEncontrado = null
                mensagemBusca = "Digite um número de ID."
            } else {
                escopo.launch {
                    try {
                        livroEncontrado = ApiClient.livroApi.buscarLivro(id)
                        mensagemBusca = ""
                    } catch (e: retrofit2.HttpException) {
                        livroEncontrado = null
                        mensagemBusca = "Livro não encontrado."
                    } catch (e: Exception) {
                        livroEncontrado = null
                        mensagemBusca = "Erro de conexão: ${e.message}"
                    }
                }
            }
        })
```

```
        livroEncontrado?.let { CardLivro(livro = it) }

        if (mensagemBusca.isNotEmpty()) {
            Text(
                text = mensagemBusca,
                modifier = Modifier.padding(16.dp),
            )
        }

        Text(
            text = "Todos os livros",
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier.padding(start = 16.dp, top = 16.dp),
        )
        CarrosselDeLivros(livros)

        FormularioCadastro(onCadastrar = { request ->
            escopo.launch {
                try {
                    val novo = ApiClient.livroApi.cadastrarLivro(request)
                    mensagemCadastro = "Livro cadastrado com sucesso! ID: ${novo.id}"
                    carregarLivros()
                } catch (e: retrofit2.HttpException) {
                    mensagemCadastro = "Dados inválidos. Confira o título (obrigatório, até 100
caracteres)."
                } catch (e: Exception) {
                    mensagemCadastro = "Erro de conexão: ${e.message}"
                }
            }
        })

        if (mensagemCadastro.isNotEmpty()) {
            Text(
                text = mensagemCadastro,
                modifier = Modifier.padding(16.dp),
            )
        }
    }
}
```

Os estados (a "memória" da tela)

Estado	Para que serve
`livros`	A lista que vem do `GET /livros`. Começa **vazia**. Quando muda, o carrossel se redesenha.
`livroEncontrado`	O livro achado na busca. `Livro?` (com `?`) significa que **pode ser** `null` (nenhum).
`mensagemBusca`	Texto de erro da busca ou do carregamento da lista.

Estado	Para que serve
`mensagemCadastro`	Texto de sucesso ou erro do cadastro.
`escopo = rememberCoroutineScope()`	Permite **iniciar chamadas de rede** (coroutines) a partir de um clique.

`carregarLivros()`

Trecho	Explicação
`suspend fun`	Função de rede que pode esperar. Fica dentro da tela para poder mudar os estados.
`try { ... } catch (e: Exception) { ... }`	Tenta buscar. Se algo falhar (servidor desligado, sem conexão), **não quebra o app** e mostra a mensagem.
`livros = ApiClient.livroApi.listarLivros()`	Faz o `GET /livros` e guarda o resultado.

`LaunchedEffect(Unit) { carregarLivros() }` Executa o bloco **uma única vez, quando a tela abre**. É por isso que os livros aparecem sozinhos ao abrir o app.

A busca (`onBuscar`)

Trecho	Explicação
`texto.toLongOrNull()`	Tenta transformar o texto em número. Se não der (vazio ou letras), devolve `null`.
`if (id == null)`	Texto inválido: mostra "Digite um número de ID." **sem chamar o Backend**.
`escopo.launch { ... }`	Inicia a chamada de rede **sem travar a tela**.
`ApiClient.livroApi.buscarLivro(id)`	Faz o `GET /livros/{id}`.
`catch (e: retrofit2.HttpException)`	O Backend respondeu com **erro HTTP** (aqui, o **404**). Mostra "Livro não encontrado.".
`catch (e: Exception)`	Qualquer outro problema (por exemplo, **sem conexão**): "Erro de conexão".

Mostrar o resultado

Trecho	Explicação
`livroEncontrado?.let { CardLivro(livro = it) }`	Se `livroEncontrado` **não for** `null`, desenha o card. Se for `null`, não desenha nada.
`if (mensagemBusca.isNotEmpty())`	Só mostra o texto quando há mensagem.

O cadastro (`onCadastrar`)

Trecho	Explicação
`ApiClient.livroApi.cadastrarLivro(request)`	Faz o `POST /livros` enviando o `LivroRequest`.
`"... ID: ${novo.id}"`	O Kotlin coloca o valor dentro do texto com `${...}`.

Trecho	Explicação
`carregarLivros()`	Depois do POST, **faz o GET de novo** para o carrossel mostrar o livro novo.
`catch HttpException`	O Backend recusou (**400**, por exemplo título vazio).
`catch Exception`	Falha de conexão.

O layout geral: `Column(modifier = Modifier.verticalScroll(rememberScrollState()))` coloca tudo em uma coluna **com rolagem**, para o formulário caber em telas pequenas.

7.8 `PreviewCarrosselDeLivros`

```
@Preview(showBackground = true)
@Composable
fun PreviewCarrosselDeLivros() {
    CarrosselDeLivros(
        listOf(
            Livro(1, "O Senhor dos Anéis", true, "aneis"),
            ...
        )
    )
}
```

`@Preview` serve **somente para o Android Studio mostrar uma prévia** do carrossel com dados de exemplo. Não faz parte do app que roda.

---

8. O que acontece quando o usuário usa o app

Abrir o app

1. A `MainActivity` chama `TelaBiblioteca()`.
2. A lista `livros` começa vazia, então o carrossel aparece vazio.
3. O `LaunchedEffect` chama `carregarLivros()`, que faz o `GET /livros`.
4. A resposta chega, `livros` muda, e o carrossel se **redesenha sozinho** com os 4 livros.

Cadastrar um livro

1. O usuário digita o título, marca a disponibilidade e escolhe a capa.
2. Toca em **Cadastrar**. O formulário monta o `LivroRequest` e chama `onCadastrar`.
3. A `TelaBiblioteca` faz o `POST /livros`.
4. O Backend valida, salva e responde **201** com o livro e o id.
5. O app mostra "Livro cadastrado com sucesso! ID: 5" e chama `carregarLivros()`, e o carrossel passa a ter o livro novo.

Buscar por ID

1. O usuário digita `3` e toca em **Buscar**.

2. O app faz `GET /livros/3`, o Backend responde **200** e o card do livro aparece.
3. Se digitar `99`, o Backend responde **404** e a mensagem "Livro não encontrado." aparece.

---

9. Backend x Mobile — o que cada lado faz

Assunto	Backend (Spring Boot)	Mobile (Kotlin)
Representar o livro	`Livro` (Entity), `LivroDTO`	`Livro` (data class)
O que se envia no cadastro	`LivroRequest` (com validação)	`LivroRequest` (sem validação)
Quem valida o título	**O Backend** (`@NotBlank`, `@Size`)	O app só trata a resposta 400
Onde fica a imagem da capa	Não fica (só o código)	`res/drawable`
Quem gera o `id`	**O banco**, no Backend	O app só recebe
Como trata o "não achei"	Exception + 404	`HttpException` + mensagem na tela
Formato da conversa	JSON (Jackson)	JSON (Gson)

---

10. Perguntas que o professor pode fazer (com resposta curta)

1. **O que é o Retrofit?** Biblioteca que transforma uma interface Kotlin em chamadas HTTP.
2. **Para que serve o Gson?** Converte JSON em objetos Kotlin e o contrário.
3. **O que é `suspend`?** Função que pode demorar (rede) e roda em uma coroutine, sem travar a tela.
4. **Por que o `usesCleartextTraffic`?** O Backend de estudo usa HTTP, e o Android bloqueia HTTP por padrão.
5. **Por que o app usa `127.0.0.1`?** No emulador, esse endereço é o próprio emulador. O ADB Reverse encaminha a porta 8089 para o computador.
6. **O que é `remember { mutableStateOf }`?** Um valor que a tela lembra e que, quando muda, redesenha a tela.
7. **O que o `LaunchedEffect(Unit)` faz?** Roda uma vez quando a tela abre; aqui, carrega os livros.
8. **Por que `Livro?` com interrogação?** Porque o valor pode ser `null`, ou seja, ainda não existe livro encontrado.
9. **Por que o app chama o GET de novo depois do POST?** Para o carrossel mostrar o livro recém-cadastrado.
10. **Onde ficam as imagens?** Dentro do app, em `res/drawable`. O Backend envia só o código.
11. **O que acontece com o Backend desligado?** O `catch (e: Exception)` mostra "Erro de conexão" e o app não quebra.

---

11. Roteiro sugerido para a apresentação (5 a 8 minutos)

1. **Contexto:** o app consome a API do Backend (a mesma da apresentação anterior).
2. **Mostrar a tela** no emulador: título, busca, carrossel e formulário.
3. **Mostrar o código na ordem:** `LivroModels.kt` → `LivroApi.kt` → `MainActivity.kt` (`CardLivro`, `BuscaLivro`, `FormularioCadastro`, `TelaBiblioteca`).
4. **Demonstração:**

5. Abrir o app: os 4 livros aparecem (GET).
6. Buscar o ID 3 (GET por ID) e o ID 99 ("Livro não encontrado.", 404).
7. Cadastrar "O Pequeno Príncipe" com a capa do Pequeno Príncipe (POST).
8. **Fechar:** o app só mostra e envia; quem valida e guarda é o Backend.

Checklist antes de apresentar

☐ Reiniciar o Backend (para ter só os 4 livros iniciais).
☐ Confirmar que o Backend está na porta **8089**.
☐ Conferir o encaminhamento: `adb reverse --list` deve mostrar `tcp:8089 tcp:8089`; se não mostrar, rodar `adb reverse tcp:8089 tcp:8089`.
☐ Abrir o app pelo ícone, no emulador.
☐ Ensaiar o cadastro uma vez e **reiniciar o Backend de novo** para o ID do Pequeno Príncipe voltar a ser 5.